

Математическое моделирование физических процессов

Лектор: Горюнов Алексей Германович

Преподаватель (ПР, ЛБ): Суханов Евгений Андреевич

В основном рассматривается Python, поэтому всё что связано с Matlab смотреть в [лекциях](#).

Лекция 1. Введение в курс математического моделирования физических процессов

1.1 Введение в курс

Моделирование – это научный прием, метод изучения, познания реального окружающего мира.

- Оригинал – реальный изучаемый объект (физическая система, процесс, явление)
- Модель – физический или абстрактный объект. **ИМИТИРУЕТ** важные для поставленной цели свойства оригинала

Математическое моделирование предполагает описание исследуемых явлений, процессов, систем различной физической природы языком **математических соотношений**. [Всё, что на ПК, сюда попадает](#)

Классификация модели по "абстракции":

- Материальная
 - Геометрическая
 - Физическая
 - Аналоговая
- Абстрактная
 - Мнемоническая
 - Вычислительная ! [одно и тоже !](#)
 - Компьютерная
 - Математическая:

Признак	МатМодель
Характер отображения свойств объекта	Структурные, Функциональные
Иерархический уровень	Микро-, Макро-, Метауровень
Степень детализации описания внутри одного уровня	Полные, Макромодели
Способ представления свойств объекта	Аналитические, Алгоритмические, Имитационные
Способ получения	Теор., Эмпир.
...	...

Аналитическая модель – математическая модель, представляющая собой совокупность **аналитических** выражений и зависимостей, позволяющих оценивать те или иные свойства моделируемого объекта. [Аппроксимация по значениям, таблицам](#)

Имитационная модель – универсальное средство исследования сложных систем, представляющее собой **логико-алгоритмическое описание** поведения отдельных элементов системы и правил их взаимодействия, отображающих последовательность событий, возникающих в моделируемой системе. **Всё что связано с частицами; метод Монте-Карло**

1.1.4 Подход к математическому моделированию процессов

7



Info >

Записал определение, сделал ссылочку на рецензируемое
Схему нужно пройти от и до для выполнения курсового проекта

1.2 Введение в Matlab. Язык Matlab: константы и переменные

Константа – это предварительно определенное числовое или символьное значение, представленное уникальным именем.

Числа являются безымянными числовыми константами. Другие виды констант в MATLAB принято назвать системными переменными, поскольку, с одной стороны, они задаются системой при ее загрузке, а с другой – могут переопределяться. Основные системные переменные, применяемые в системе MATLAB, указаны ниже:

- **i** или **j** – мнимая единица;
- **pi** – число π – 3.1415926...;
- **eps** – погрешность операций над числами с плавающей точкой (2^{-52});
- **realmin** – наименьшее число с плавающей точкой (2^{-1022});
- **realmax** – наибольшее число с плавающей точкой (2^{1023});
- **inf** – значение машинной бесконечности;
- **ans** – переменная, хранящая результат последней операции и обычно вызывающая его отображение на экране дисплея;
- **NaN** – указание на нечисловой характер данных (Not-a-Number).

Основные типы данных: числовые, logical, char, cell, struct, table и function_handle.

...

1.3 Введение в Python для задач моделирования

Переменная – это область памяти, в которой хранятся наши данные. В переменную задается имя и значение, например:

```
test = 1
value = "2"
result = 3.0
test2 = True
```

Каждая переменная содержит тип данных, например: `float`, `int`, `str`, `bool`, `complex` и т.д. Арифметические операторы используются с числовыми значениями для выполнения общих математических операций: `+`, `-`, `*`, `/`, `%`, `**`, `//`.

Операторы присваивания используются для присваивания значений переменным: `=`, `+=`, `-=`, `*=`, `/=`, `%=`, `//=`, `**=`, `&=`, `|=`, `^=`, `>=`, `<=`.

Операторы сравнения, логические: `==`, `!=`, `>`, `<`, `>=`, `<=`, `and`, `or`, `not`.

```
# Условный оператор
if condition:
    <indented statement 1>
    <indented statement 2>
elif condition:
    <indented statement 3>
else:
    <indented statement 4>

# Циклы: оператор for
for <variable> in <sequence>:
    <body of for>
else:
    <body of else>

# Циклы: оператор while
while test_expression:
    body of while
```

Лекция 2. Работа с данными в Matlab и Python

2.1 Операции с массивами/матрицами в Python

Доступ к элементам матриц осуществляется при помощи двух индексов - номеров строки и столбца, заключенных в квадратные скобки для каждого индекса отдельно. [БАЗА](#). При этом нумерация идёт с 0

При помощи библиотеки NumPy вектора/матрицы записываются так:

```
import numpy as np

# Матрицы и действия над ними
A = np.array([1,2,3],[1,2,3],[6,-2,8])
B = np.array([8,8,8],[1,2,3],[-6,2,-8])
```

```

S = A + B          # Сумма
P = np.dot(A,B)    # Умножение          # Аналогично: A@B
G = A * 3          # Умножение на число
T = np.transpose(B) # Транспонирование
H = np.linalg.inv(B) # Обратная матрица

# Удаление строк и столбцов
X = A[0,:]         ''' Вывод: [1,2,3] '''
Y = A[2,:]         ''' Вывод: [6,-2,8] '''
Z = A[:,1]         ''' Вывод: [2,2,-2] '''

# Создание матриц специального вида
Zer = np.zeros(3)
Eye = np.eye(3)
One = np.ones(3)

```

...

"""Функция read() используется для чтения содержимого файла после открытия его в режиме чтения (r).

- file = объект файла
- size = количество символов, которые нужно прочитать. Если не указать, то файл читается целиком. """

file.read(size)

""" Функция write() используется для записи в файлы Python, открытые в режиме записи. Если попытаться открыть файл, которого не существует, в этом режиме, тогда будет создан новый. """

file.write(string)

"""Метод close(). После открытия файла в Python его нужно закрыть. Таким образом освобождаются ресурсы и убирается мусор. Python автоматически закрывает файл, когда объект присваивается другому файлу."""

2.4 Функции в Python

Пока в Matlab существуют m-файлы, анонимные, вложенные функции, подфункции и операторы дескриптора функции, в Python есть просто функции))). `def`, `lambda`... которые могут ещё использоваться для создания классов в ООП.

```

''' python
# Стандартные функции
def name_functions(var1,var2,... or nothing):
    '''docstring'''
    <instruction 1>
    <instruction 2>
    <instruction 3>
    return expression

# Лямбда-функции (анонимные функции)
x = lambda var: expression

```

Лекция 3. Визуализация вычислений в Matlab и Python

3.1 Графическая визуализация вычислений в Python

`matplotlib` – библиотека, предназначенная для отображения двумерных и 3D-графиков, максимально точно воссоздает среду MATLAB в плане графического интерфейса и синтаксической формы. [12](#)

3.2 Построение двумерных графиков в Python

[Старт в Matplotlib](#)

Пример 1

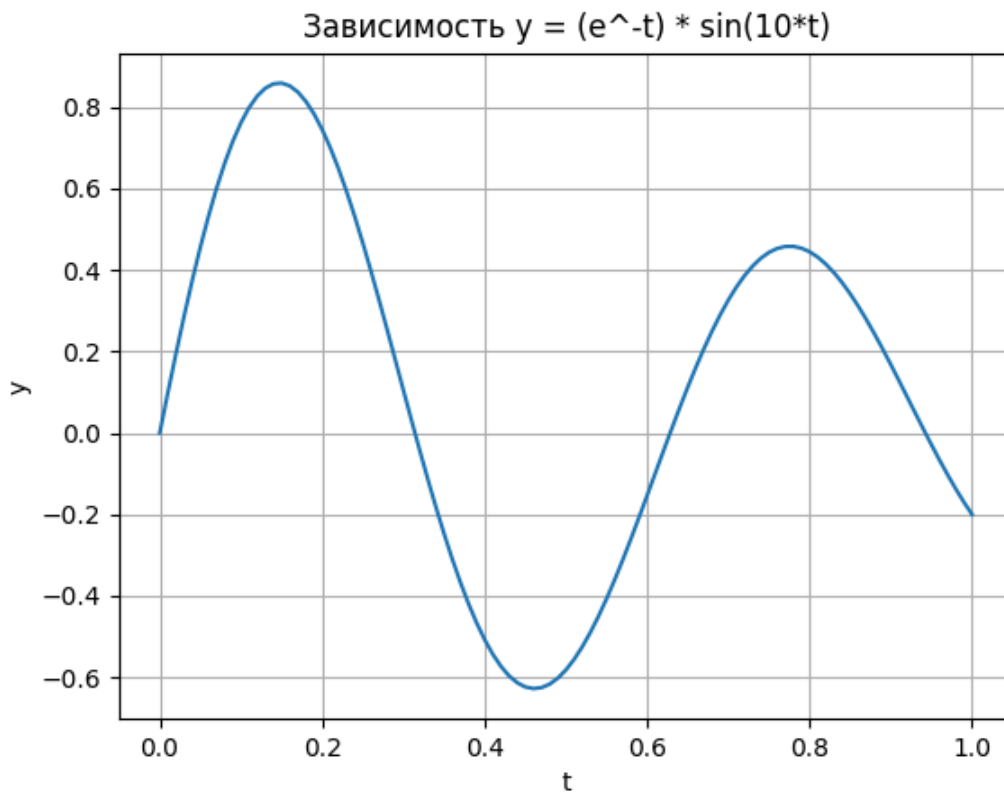
График функции $y(t) = e^{-t} \cdot \sin(10 \cdot t)$, где аргумент t меняется в интервале $[0, 1]$ с шагом 0,01.

```
import matplotlib.pyplot as plt
import numpy as np

t = np.zeros(101)
y = np.zeros(101)
y[0] = np.exp(-t[0]) * np.sin(10*t[0])

for i in range(1,101):
    t[i] = t[i-1] + 0.01
    y[i] = np.exp(-t[i]) * np.sin(10*t[i])

# Построение графика
plt.title("Зависимость y = (e^-t) * sin(10*t)") # заголовок
plt.xlabel("t") # ось абсцисс
plt.ylabel("y") # ось ординат
plt.grid() # включение отображение сетки
plt.plot(t, y) # построение графика
plt.show()
```



Пример 2

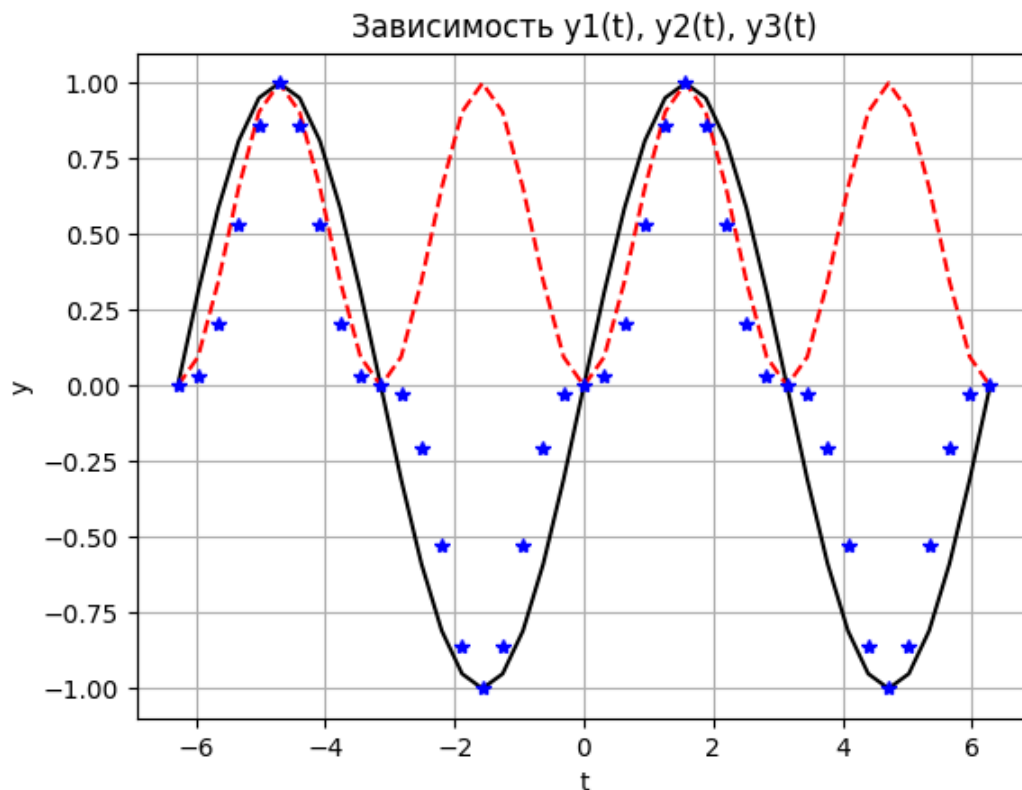
График функции $y_1(t) = \sin(t)$, $y_2(t) = \sin^2(t)$ и $y_3(t) = \sin^3(t)$ при изменении t в интервале $[-2\pi, 2\pi]$ с шагом $0,1\pi$.

```
import matplotlib.pyplot as plt
import numpy as np

t = np.zeros(41)
t[0] = -2*np.pi
y1 = np.zeros(41)
y2 = np.zeros(41)
y3 = np.zeros(41)
y1[0] = np.sin(t[0])

for i in range(1,41):
    t[i] = t[i-1] + 0.1*np.pi
    y1[i] = np.sin(t[i])
    y2[i] = pow(np.sin(t[i]),2)
    y3[i] = pow(np.sin(t[i]),3)

# Построение графика
plt.title("Зависимость y1(t), y2(t), y3(t)")
plt.xlabel("t") # ось абсцисс
plt.ylabel("y") # ось ординат
plt.grid() # включение отображение сетки
plt.plot(t, y1, "k-", t, y2, "r--", t, y3, "b*")
plt.show()
```



Пример 3

График функции $z(x, y) = x + y^3$, где переменные x и y изменяются на интервале $[-10, 10]$ с шагом 0,5. Для построения графика используем непрерывные линии черного цвета.

```
import matplotlib.pyplot as plt
import numpy as np

x = np.zeros(41)
x[0] = -10
for i in range(1, 41):
    x[i] = x[i-1] + 0.5

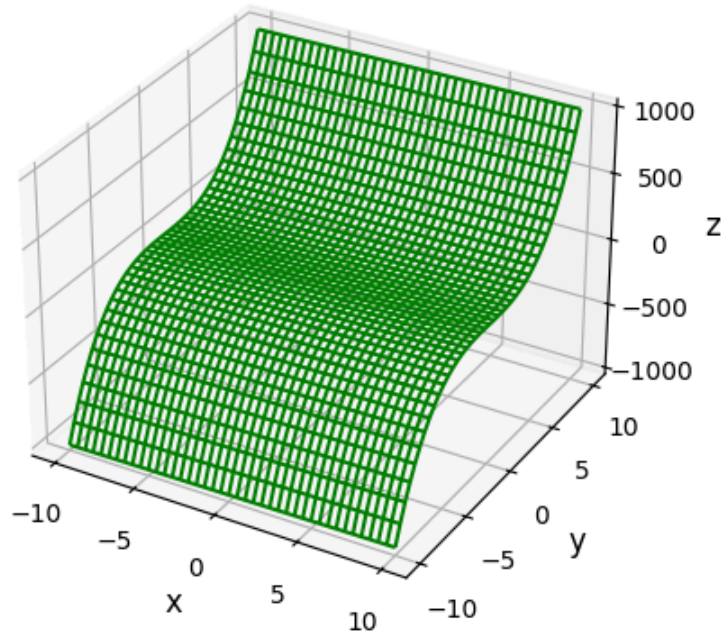
y = x

# function for z axis
def f(x, y):
    return x + pow(y, 3)

X, Y = np.meshgrid(x, y)
Z = f(X, Y)

fig = plt.figure()
ax = plt.axes(projection='3d')
ax.plot_wireframe(X, Y, Z, color='green')
ax.set_title("z(x,y) = x + y^3")
ax.set_xlabel('x', fontsize=12)
ax.set_ylabel('y', fontsize=12)
ax.set_zlabel('z', fontsize=12)
plt.show()
```

$$z(x,y) = x + y^3$$



Пример 4

График функции $z = \frac{\sin(x)}{x^2+y^2+0,3}$, где x и y изменяются в интервале $[-4, 4]$ с шагом 0,1.

```
import matplotlib.pyplot as plt
import numpy as np

x = np.zeros(81)
x[0] = -4
for i in range(1,81):
    x[i] = x[i-1] + 0.1

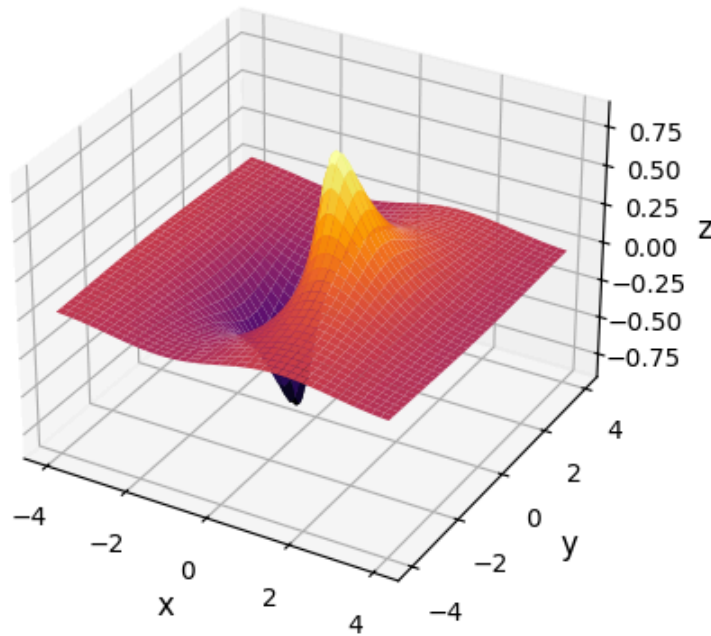
y = x

# function for z axis
def f(x, y):
    return np.sin(x) / (pow(x,2) + pow(y,2) + 0.3)

X, Y = np.meshgrid(x, y)
Z = f(X, Y)

fig = plt.figure()
ax = plt.axes(projection='3d')
ax.plot_surface(X, Y, Z, cmap='inferno')
ax.set_title("z(x,y)=sin(x)/(x^2 + y^2 + 0.3)")
ax.set_xlabel('x', fontsize=12)
ax.set_ylabel('y', fontsize=12)
ax.set_zlabel('z', fontsize=12)
plt.show()
```

$$z(x,y)=\sin(x)/(x^2 + y^2 + 0.3)$$



```
3>
return expression
```

Лекция 4. Методы восстановления эмпирических зависимостей

4.1 Интерполяция, экстраполяция

Интерполяция – способ нахождения промежуточных значений величины по имеющемуся дискретному набору известных значений. Нахождение приближенной функции. $x_0, x_1, \dots, x_n$ – узлами интерполяции.

Графически задача интерполирования заключается в том, чтобы построить такую интерполирующую функцию, которая бы проходила через все узлы интерполирования.

Вид канонического полинома степени n :

$$P_n(x) = a_0 + a_1x + a_2x^2 + \dots + a_{n-1}x^{n-1} + a_nx^n \quad (1)$$

Выбор многочлена степени n основан на том факте, что через $n+1$ точку проходит единственная кривая степени n . На основе (1) и данных интерполяции составим СЛАУ, решая её, можно найти коэффициенты интерполяционного полинома $a_0, a_1, \dots, a_n$.

Экстраполяция – это индуктивное нахождение по ряду данных значений функции других её значений, находящихся вне этого ряда. **Т.е. приближенное значение $f(x)$, где x лежит вне $[x_0, x_n]$.**

Экстраполяция функции – продолжение функции за пределы её области определения, при котором продолженная функция (как правило, аналитическая) принадлежит заданному классу функций.

4.2 Интерполяционный полином Лагранжа

Интерполяционный полином Лагранжа имеет вид:

$$P_n(x) = \sum_{i=0}^n x_i \cdot L_n(x), \quad (2)$$

где $L_n(x)$ – множитель Лагранжа, определяемый выражением:

$$L_n(x) = \frac{(x - x_0) \cdots (x - x_{i-1})(x - x_{i+1}) \cdots (x - x_n)}{(x_i - x_0) \cdots (x_i - x_{i-1})(x_i - x_{i+1}) \cdots (x_i - x_n)} = \prod_{\substack{k=0 \\ k \neq i}}^n \frac{(x - x_k)}{(x_i - x_k)} \quad (3)$$

В свернутом виде:

$$P_n(x) = \sum_{i=0}^n y_i \left(\prod_{\substack{k=0 \\ k \neq i}}^n \frac{x - x_k}{x_i - x_k} \right) \quad (4)$$

4.3 Интерполяционные полиномы Ньютона

Если узлы интерполяции равноотстоящие по величине, так что выполняется условие:

$$x_{i+1} - x = h = const, \quad (5)$$

где h – шаг интерполяции, т.е. $x_i = x_0 + n \cdot h$, то интерполяционный многочлен можно записать в форме, предложенной Ньютоном. Интерполяционные полиномы Ньютона как правило используют, если точки интерполирования находится в начале таблицы – **первая интерполяционная формула** Ньютона или в конце таблицы – **вторая формула**.

Интерполирующий полином:

$$P_n(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \cdots + a_n(x - x_0) \cdots (x - x_{n-1}) \quad (6)$$

Коэффициенты $a_0, a_1, \dots, a_n$ находятся из условия $P_n(x_i) = y_i$.

Определим a_0 , полагая $x = x_0$:

$$a_0 = P_n(x_0) = y_0 \quad (7)$$

Для определения a_2 , полагая $x = x_2$, получим:

$$\begin{aligned} P_n(x_2) = y_2 = y_0 + \frac{\Delta y_0}{h}(x_2 - x_0) + a_2(x_2 - x_0)(x_2 - x_1) &= y_0 + 2\Delta y_0 + a_2 \cdot 2 \cdot h^2, \\ a_2 = \frac{y_2 - y_0 - 2\Delta y_0}{2h^2} = \frac{y_2 - y_0 - 2y_1 + 2y_0}{2h^2} = \frac{y_2 - 2y_1 + y_0}{2h^2} &= \frac{(y_2 - y_1) - (y_1 - y_0)}{2h^2} = \\ &= \frac{\Delta y_1 - \Delta y_0}{2h^2} = \frac{\Delta^2 y_0}{2! h^2}. \end{aligned} \quad (8)$$

Общая формула для нахождения всех коэффициентов имеет вид:

$$\begin{aligned} a_i &= \frac{\Delta^i y_0}{i! h^i}, \\ \Delta^i y_0 &= \Delta^{i-1} y_1 - \Delta^{i-1} y_0, \\ \Delta^i y_1 &= \Delta^{i-1} y_2 - \Delta^{i-1} y_1 \end{aligned} \quad (9)$$

где $i = 1 \dots n$.

$\Delta^2 y_0 = \Delta y_1 - \Delta y_0,$	$\Delta^2 y_1 = \Delta y_2 - \Delta y_1,$
$\Delta y_0 = y_1 - y_0,$	$\Delta y_1 = y_2 - y_1.$

В результате (6) примет вид:

$$P_n(x) = y_0 + \frac{\Delta y_0}{1! h} (x - x_0) + \frac{\Delta^2 y_0}{2! h^2} (x - x_0)(x - x_1) + \dots$$

$$\dots + \frac{\Delta^n y_0}{n! h^n} (x - x_0)(x - x_1) \dots (x - x_{n-1}). \quad (10)$$

бббrrrrrr... чё так страшно... плак, плак, т.к это ещё не всё
... (типо решаем) ...

4.4 Интерполяция сплайнами

rrrrrrrr

4.5 Аппроксимация. Метод наименьших квадратов

rrrrrrrr

4.6 Математическая обработка данных в Python

Итак первые 3 пункта тут будут о том, как работать с файлами и данными из них, если вам нужен сам код для интерполяции смотрите начиная с четвёртого пункта.

Получение исходной таблицы данных

$$\begin{cases} y = 0.5 \cdot \sin(x) + 2 \cdot \text{rand}, \\ x \in [0; 10], \text{ шаг } 1 \end{cases}$$

```
import matplotlib.pyplot as plt
import numpy as np

# --- Часть 1: Получение исходной таблицы данных ---

x = np.zeros(11)
y = np.zeros(11)
float_rn = np.zeros(11)

# Заполнение вектора x от 0 до 10 с шагом 1
for i in range(0, 10):
    x[i+1] = x[i] + 1

# Заполнение вектора случайных чисел и вычисление y
for i in range(0, 11):
    float_rn[i] = np.random.rand()
    y[i] = 0.5 * np.sin(x[i]) + 2 * float_rn[i]

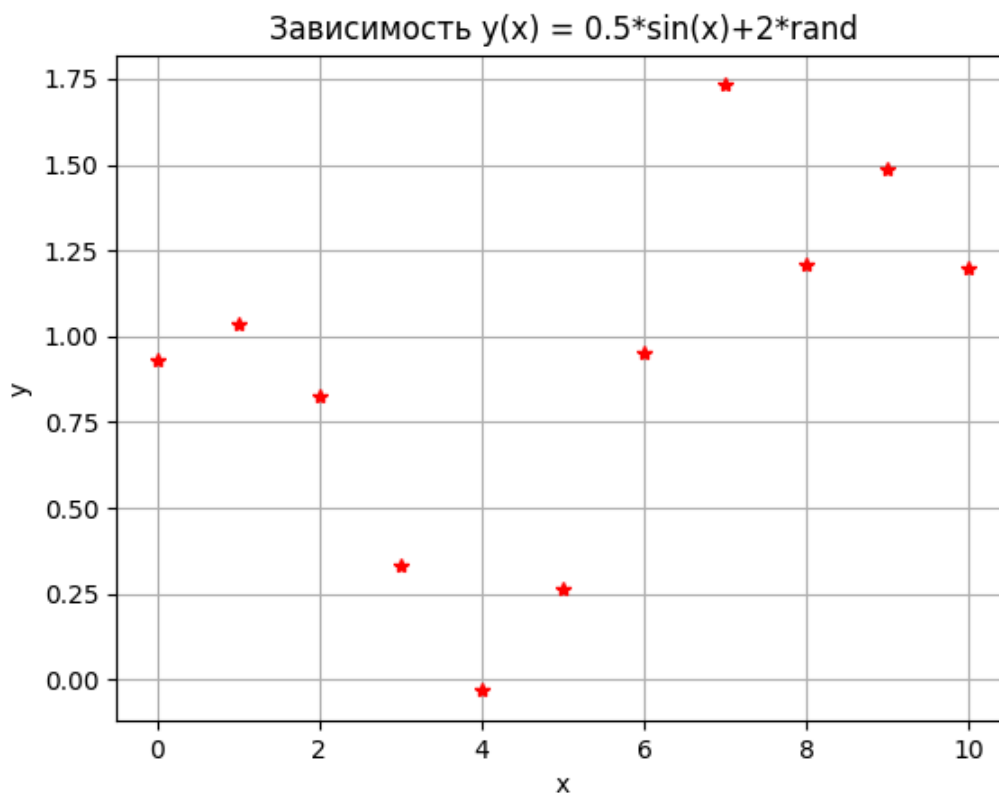
print("Вектор значений x: \n", x)
print("Вектор сл.значений от 0 до 1: \n", float_rn)
print("Вектор значений y(x): \n", y)

# --- Часть 2: Построение графика ---
```

```
# Построение графика
plt.title("Зависимость  $y(x) = 0.5 \cdot \sin(x) + 2 \cdot \text{rand}$ ") # заголовок
plt.xlabel("x") # ось абсцисс
plt.ylabel("y") # ось ординат
plt.grid()      # включение отображение сетки
plt.plot(x, y, "r*") # построение графика (красные звездочки)
plt.show()
```

```
'''
OUTPUT:
```

```
Вектор значений x: [ 0.  1.  2.  3.  4.  5.  6.  7.  8.  9. 10.]
Вектор сл.значений от 0 до 1: [0.46449122 0.30669835 0.18620595 0.13223051 0.17332797
0.37172477 0.54480501 0.70161823 0.35709033 0.64016975 0.73477446]
Вектор значений y(x): [ 0.92898244  1.0341322  0.82706062  0.33502103 -0.03174531
 0.26398739  0.94990228  1.73172976  1.20885978  1.48639874  1.19753837]
'''
```



Сохранение данных в файл

```
# Сохранение исходных данных
f = open("data.txt", "w")
for i in range(0,11):
    f.write("%f" % x[i])
    f.write(" ")
    f.write("%f \n" % y[i])

f.close()
```

Чтение данных из текстового файла

[Считываем числовые данные из файла на Python](#)

```
import matplotlib.pyplot as plt
import numpy as np
x = np.zeros(11)
y = np.zeros(11)

# Чтение исходных данных
# -----
import re
file = open("data.txt")
values = file.read().split("\n")
data = []
for key in values:
    value = re.findall(r"[-+]?[d*\.]d+|\d+", key)

    if value != []:
        data.append(value)
# -----

for i in range(0,11):
    x[i] = data[i][0]
    y[i] = data[i][1]

print("Считанные данные из файла: \n", data)
print("Вектор значений x: \n",x)
print("Вектор значений y(x): \n",y)

'''
OUTPUT:

Считанные данные из файла:
    тут дохуя написано, если интересно см. ЛК4 или Ctrl+C, +V в CMD
'''
```

Построить интерполяционный многочлен Лагранжа

```
import matplotlib.pyplot as plt
import numpy as np

def lagrang(xf, xe, ye, ne):
    Pn = 0
    for i in range(0, ne):
        mult = 1
        for k in range(0, ne):
            if(k != i):
                mult = mult * ((xf - xe[k])/(xe[i] - xe[k]))

        Pn = Pn + ye[i] * mult

    return Pn
```

```

# Чтение исходных данных
# -----
import re
file = open("data.txt")
values = file.read().split("\n")
data = []
for key in values:
    value = re.findall(r"[-+]?[d*]\.d+|\d+", key)

    if value != []:
        data.append(value)
# -----
x = np.zeros(11)
y = np.zeros(11)
for i in range(0, 11):
    x[i] = data[i][0]
    y[i] = data[i][1]

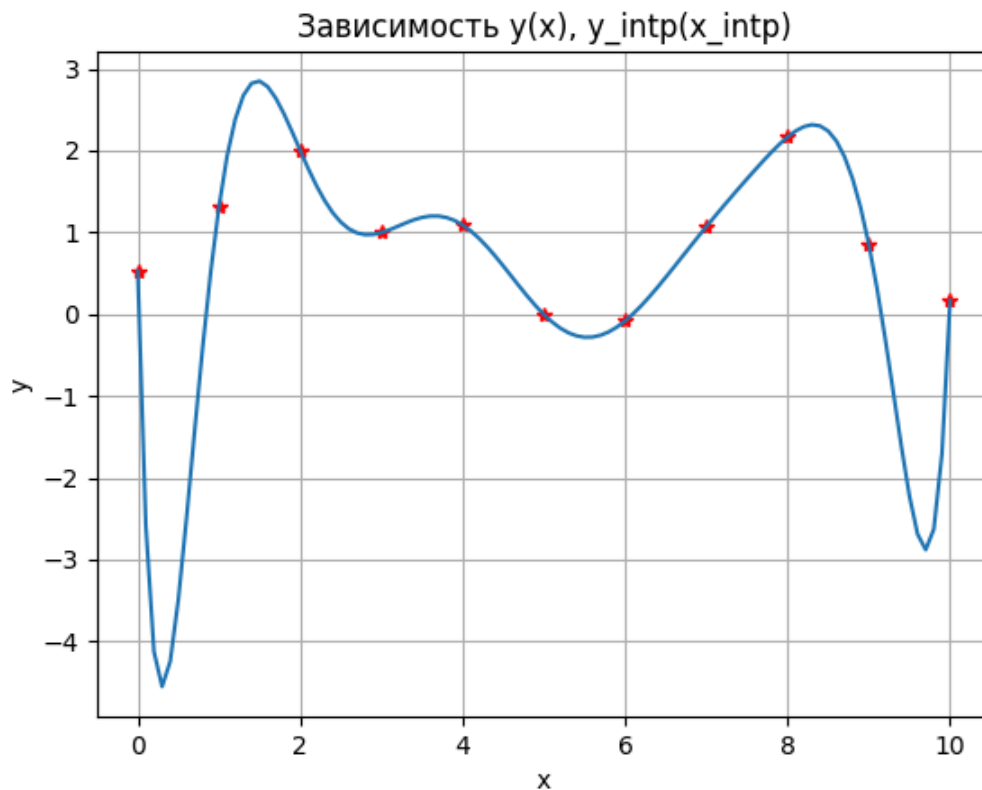
print("Считанные данные из файла: \n", data)
print("Вектор значений x: \n", x)
print("Вектор значений y(x): \n", y)

x_intp = np.zeros(101)
n_x_intp = np.size(x_intp)
for i in range(0, n_x_intp-1):
    x_intp[i+1] = x_intp[i] + 0.1

y_intp = np.zeros(n_x_intp)
n = np.size(x)
for i in range(0, n_x_intp):
    y_intp[i] = lagrang(x_intp[i], x, y, n)

# Построение графика
plt.title("Зависимость y(x), y_intp(x_intp)") # заголовок
plt.xlabel("x") # ось абсцисс
plt.ylabel("y") # ось ординат
plt.grid() # включение отображение сетки
plt.plot(x, y, "r*", x_intp, y_intp) # построение графика
plt.show()

```



Построить линейную интерполяцию

⚠ Внимание >

К сожалению в ЛК представленный код идёт с использованием встроенных функций в библиотеки NumPy и SciPy. Тут его предоставлю пока, что в дальнейшем он будет доработан кодом собственным по реализации линейной интерполяции, кубического сплайна и МНК.

```
import matplotlib.pyplot as plt
import numpy as np

# Чтение исходных данных
# -----
import re
file = open("data.txt")
values = file.read().split("\n")
data = []
for key in values:
    value = re.findall(r"[-+]?[d*]\.d+|\d+", key)

    if value != []:
        data.append(value)

# -----
x = np.zeros(11)
y = np.zeros(11)
for i in range(0,11):
    x[i] = data[i][0]
    y[i] = data[i][1]
```

```

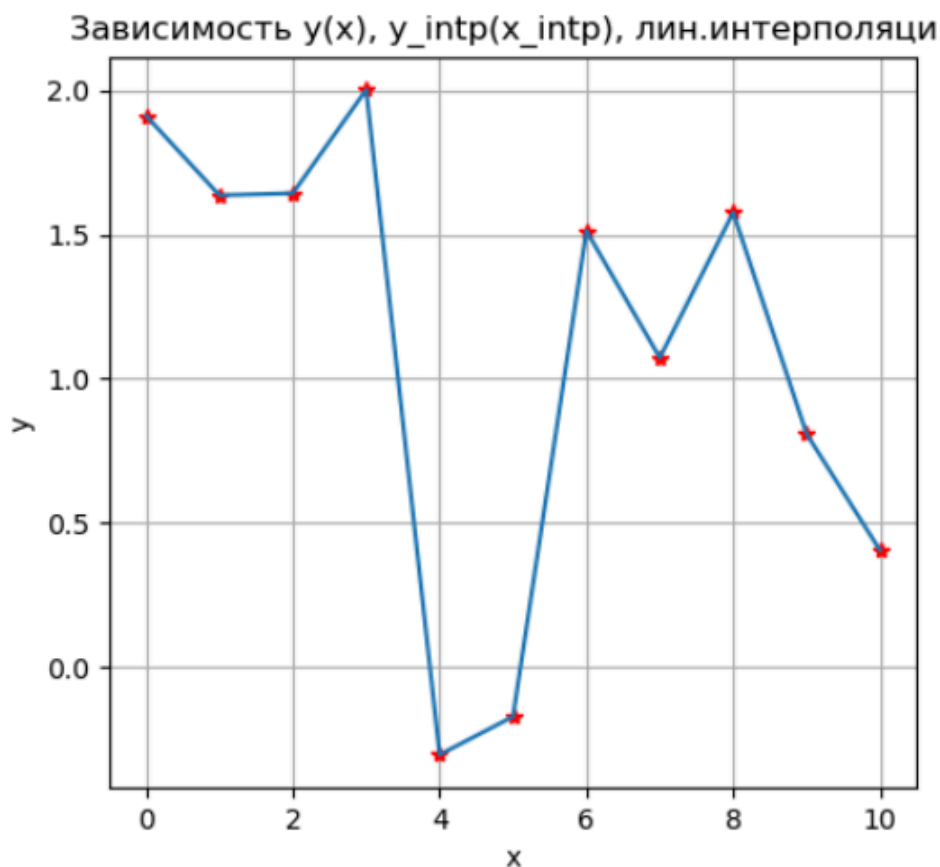
print("Считанные данные из файла: \n", data)
print("Вектор значений x: \n", x)
print("Вектор значений y(x): \n", y)

x_intp = np.zeros(101)
n_x_intp = np.size(x_intp)
for i in range(0, n_x_intp-1):
    x_intp[i+1] = x_intp[i] + 0.1

y_intp = np.zeros(n_x_intp)

# Линейная интерполяция на основе библиотеки NumPy
y_intp = np.interp(x_intp, x, y)

```



Построить кубическую сплайн-интерполяцию

⚠ Внимание >

К сожалению в ЛК представленный код идёт с использованием встроенных функций в библиотеки NumPy и SciPy. Тут его предоставляю пока, что в дальнейшем он будет доработан кодом собственным по реализации линейной интерполяции, кубического сплайна и МНК.

```

import matplotlib.pyplot as plt
import numpy as np

# Чтение исходных данных
# -----
import re

```

```

file = open("data.txt")
values = file.read().split("\n")
data = []
for key in values:
    value = re.findall(r"[~+]?~*\.\d+|\d+", key)

    if value != []:
        data.append(value)
# -----
x = np.zeros(11)
y = np.zeros(11)
for i in range(0,11):
    x[i] = data[i][0]
    y[i] = data[i][1]

print("Считанные данные из файла: \n", data)
print("Вектор значений x: \n",x)
print("Вектор значений y(x): \n",y)

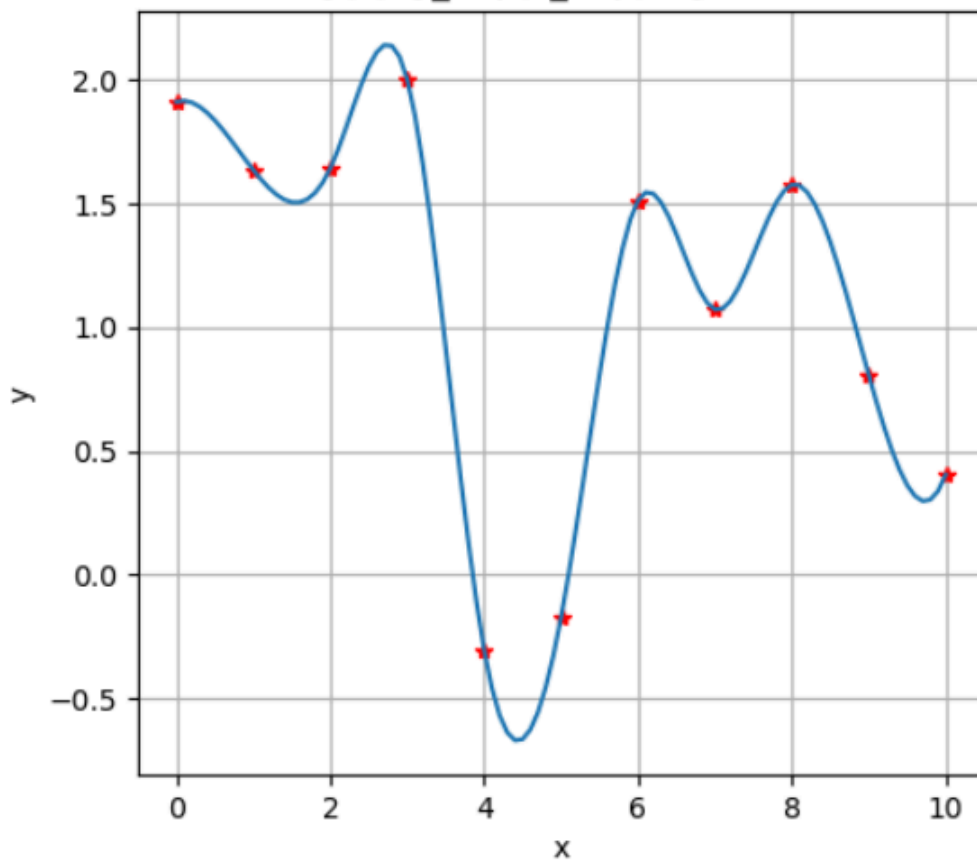
x_intp = np.zeros(101)
n_x_intp = np.size(x_intp)
for i in range(0,n_x_intp-1):
    x_intp[i+1] = x_intp[i] + 0.1

y_intp = np.zeros(n_x_intp)

# Линейная интерполяция на основе библиотеки NumPy
y_intp = np.interp(x_intp, x, y)

```

Зависимость $y(x)$, $y_{\text{intp}}(x_{\text{intp}})$, кубическая инт.-ция



Построить кубическую сплайн-интерполяцию

⚠ Внимание >

К сожалению в ЛК представленный код идёт с использованием встроенных функций в библиотеки NumPy и SciPy. Тут его предоставлю пока, что в дальнейшем он будет доработан кодом собственным по реализации линейной интерполяции, кубического сплайна и МНК.

```
import matplotlib.pyplot as plt
import numpy as np

# Чтение исходных данных
# -----
import re
file = open("data.txt")
values = file.read().split("\n")
data = []
for key in values:
    value = re.findall(r"[-+]?[d*\.\\d+|\\d+", key)

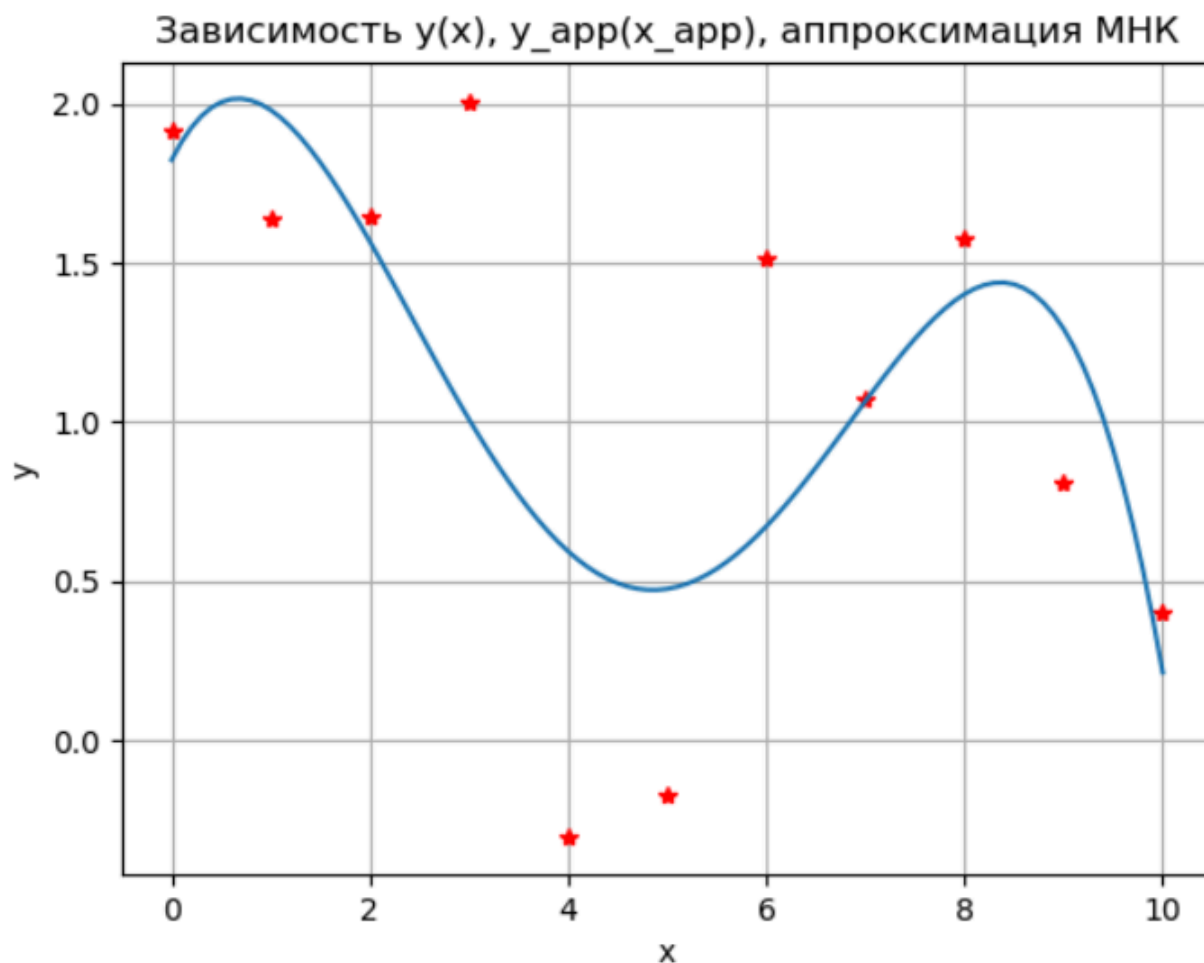
    if value != []:
        data.append(value)
# -----
x = np.zeros(11)
y = np.zeros(11)
for i in range(0,11):
    x[i] = data[i][0]
    y[i] = data[i][1]

print("Считанные данные из файла: \n", data)
print("Вектор значений x: \n",x)
print("Вектор значений y(x): \n",y)

x_intp = np.zeros(101)
n_x_intp = np.size(x_intp)
for i in range(0,n_x_intp-1):
    x_intp[i+1] = x_intp[i] + 0.1

y_intp = np.zeros(n_x_intp)

# Аппроксимация данных методом наименьших квадратов
# на основе функции полифит библиотеки NumPy
koeff = np.polyfit(x, y, 4)
print("Коэффициенты полинома", koeff)
for i in range(0, n_x_app):
    y_app[i] = koeff[0] * pow(x_app[i], 4) + koeff[1] * pow(x_app[i], 3) + koeff[2] *
pow(x_app[i], 2) + koeff[3] * pow(x_app[i], 1) + koeff[4]
```



Лекция 5. Теория численного интегрирования

5.1 Задача численного интегрирования

5.2 Квадратурные формулы прямоугольников

5.3 Квадратурная формула трапеций

5.4 Квадратурная формула Симпсона

5.5 Квадратурные формулы интерполяционного типа

5.6 Численное и символьное интегрирование в Python